

Unit V

**Derived Types, Structures and
Unions, File Handling**

Derived Types

- Enumerated Types: Declaring an Enumerated Type, Operations on Enumerated Types, Initializing Enumerated Constants.
- Enumeration or Enum in C is a special kind of data type defined by the user. It consists of constant integers or integers that are given names by a user. The use of enum in C to name the integer values makes the entire program easy to learn, understand, and maintain by the same or even different programmer.

Syntax of Enum

An enum is defined by using the 'enum' keyword in C, and the use of a comma separates the constants within.

The basic syntax of defining an enum is:

```
enum enum_name{int_const1, int_const2, int_const3, ... int_constN};
```

In the above syntax, the default value of int_const1 is 0, int_const2 is 1, int_const3 is 2, and so on. However, you can also change these default values while declaring the enum. Below is an example of an enum named cars and how you can change the default values.

```
enum cars{BMW, Ferrari, Jeep, Mercedes-Benz};
```

Here, the default values for the constants are:

BMW=0, Ferrari=1, Jeep=2, and Mercedes-Benz=3.

However, to change the default values, you can define the enum as follows:

```
enum cars{  
    BMW=3,  
    Ferrari=5,  
    Jeep=0,  
    Mercedes-Benz=1  
};
```

Enumerated Type Declaration to Create a Variable

Similar to pre-defined data types like int and char, you can also declare a variable for enum and other user-defined data types.

Here's how to create a variable for enum.

```
enum condition (true, false); //declaring the enum
```

```
enum condition e; //creating a variable of type condition
```

Suppose we have declared an enum type named condition; we can create a variable for that data type as mentioned above. We can also converge both the statements and write them as:

```
enum condition (true, false) e;
```

For the above statement, the default value for true will be 1, and that for false will be 0.

How to Create and Implement enum

Now that we know how to define an enum and create variables for it, let's look at some examples to understand how to implement enum in C programs.

Printing the Values of Weekdays

```
#include <stdio.h>

enum days{Sunday=1, Monday, Tuesday, Wednesday, Thursday, Friday,
Saturday};

int main(){
    // printing the values of weekdays
    for(int i=Sunday;i<=Saturday;i++){
        printf("%d, ",i);
    }
    return 0;
}
```

Output:

1,2,3,4,5,6,7,

In the above code, we declared an enum named days consisting of the name of the weekdays starting from Sunday. We then initialized the value of Sunday to be 1. This will assign the value for the other days as the previous value plus 1. To iterate through the enum and print the values of each day, we have created a for loop and initialized the value for i as Sunday.

Program for enum in C

```
#include<stdio.h>
enum week{Mon, Tue, Wed, Thur, Fri, Sat, Sun};
int main()
{
    enum week day;
    day = Wed;
    printf("%d",day);
    return 0;
}
```

Output:

2

STRUCTURES :

- A Structure is a collection of elements of dissimilar data types. Structures provide the ability to create user defined data types and also to represent real world data.
- Suppose if we want to store the information about a book, we need to store its name (String), its price (float) and number of pages in it(int). We have to store the above three items as a group then we can use a structure variable which collectively store the information as a book.
- Structures can be used to store the real world data like employee, student, person etc.

Contd..

Declaration :

The declaration of the Structure starts with a Key Word called Struct and ends with ; .

The Structure elements can be any built in types.

```
struct <Structure name>
```

```
{
```

```
Structure element 1;
```

```
Structure element 2;
```

```
Structure element n;
```

```
};
```


Contd..

Then the Structure Variables are declared as

```
struct < Structure name > <Var1,Var2>;
```

Eg:-

```
struct emp
```

```
{
```

```
int empno.
```

```
char ename[20]; float sal;
```

```
};
```

```
struct emp e1,e2,e3;
```

Examples for structure

- The Structure can also be declared as :

```
struct emp
```

```
{
```

```
    int empno;
```

```
    char ename[20];
```

```
    float sal;
```

```
}e1,e2,e3;
```

(or)

```
struct
```

```
{
```

```
    int empno;
```

```
    char ename[20];
```

```
    float sal;
```

```
}e1,e2,e3;
```

Initialization :

Initialization :

Structure Variables can also be initialized where they are declared.

```
struct emp
```

```
{
```

```
int empno;
```

```
char ename[20]; float sal;
```

```
};
```

```
struct emp e1 = { 123,"Kumar",5000.00};
```

- To access the Structure elements we use the `.(dot)` operator. To refer empno we should use `e1.empno`
- To refer sal we should use `e1.sal`
- Structure elements are stored in contiguous memory locations as shown below. The above Structure occupies totally 26 bytes.

<code>e1.empno</code>	<code>E1.ename</code>	<code>E1.sal</code>
123	Kumar	5000.00
2000	2002	2022

Program to illustrate the usage of a Structure.

```
#include<stdio.h>
main()
{
    struct emp
    {
        int empno;
        char ename[20];
        float sal;
    };
    struct emp e;
    printf (" Enter Employee number: \n");
    scanf("%d",&e.empno);
    printf (" Enter Employee Name: \n");
    scanf("%s",&e.ename);
    printf (" Enter the Salary: \n");
    scanf("%f",&e.sal);
    printf (" Employee No = %d", e.empno);
    printf ("\n Emp Name = %s", e.ename);
    printf ("\n Salary = %f", e.sal);
}
```

Program to read Student Details and Calculate total and average using structures

```
#include<stdio.h> main()
{
    struct stud
    {
        int rno;
        char sname[20];

        int m1,m2,m3;
    };
    struct stud s;
    int tot;
    float avg;
    printf("Enter the student roll number: \n");
    scanf("%d",&s.rno);
    printf("Enter the student name: \n");
    scanf("%s",&s.sname);
    printf("Enter the three subjects marks: \n");
    scanf("%d%d%d",&s.m1,&s.m2,&s.m3);
    tot = s.m1 + s.m2 +s.m3; avg = tot/3.0;

    printf("Roll Number : %d\n",s.rno); printf("Student Name: %s\n",s.sname);
    printf("Total Marks : %d\n",tot); printf("Average : %f\n",avg);
}
```

Program to read Item Details and Calculate Total Amount of Items

```
#include<stdio.h>
main()
{
    struct item
    {
        int itemno;
        char itemname[20]; float rate;
        int qty;
    };
    struct item i; float tot_amt;
    printf("Enter the Item Number \n");
    scanf("%d",&i.itemno); printf("Enter the Item Name \n");
    scanf("%s",&i.itemname);
    printf("Enter the Rate of the Item \n");
    scanf("%f",&i.rate);
    printf("Enter the number of %s purchased ",i.itemname);
    scanf("%d",&i.qty); tot_amt = i.rate * i.qty;
    printf("Item Number: %d\n",i.itemno); printf("Item Name: %s\n",i.itemname);
    printf("Rate: %f\n",i.rate); printf("Number of Items: %d\n",i.qty); printf("Total Amount: %f",tot_amt);
}
```

ARRAY OF STRUCTURES :

- To store more number of Structures, we can use array of Structures. In array of Structures all elements of the array are stored in adjacent memory location.

Program to illustrate the usage of Array of Structures.

```
#include<stdio.h>
main()
{
    struct book
    {
        char name[20];
        float price;
        int pages;
    };
    struct book b[10];
    int i;
    for (i=0;i<10;i++)
    {
        print("\n Enter Name, Price and Pages");
        scanf("%s%f%d", b[i].name,&b[i].price,&b[i].pages);
    }
    for (i i=0;i<10;i++)
        printf(" \n%s%f%d", b[i].name,b[i].price,b[i].pages);
}
```

UNIONS

UNIONS :

Union, similar to Structures, are collection of elements of different data types. However, the members within a union all share the same storage area within the computer's memory, whereas each member within a Structure is assigned its own unique Storage area.

Structure enables us to treat a member of different variables stored at different places in memory, a union enables us to treat the same space in memory as a number of different variables. That is, a union offers a way for a section of memory to be treated as a variable of one type on one occasion and as a different variable of a different type on another occasion.

Unions are used to conserve memory. Unions are useful for applications involving multiple members, where values need not be assigned to all of the members at any given time. An attempt to access the wrong type of information will produce meaningless results.

The union elements are accessed exactly the same way in which the structure elements are accessed using dot(.) operator.

The difference between the structure and union in the memory representation is as follows.

```
struct cx
```

```
{
```

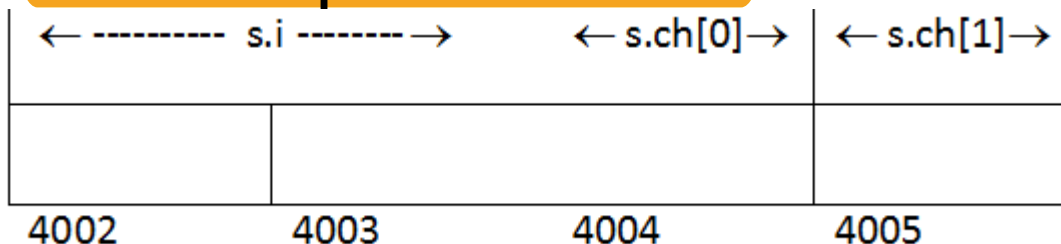
```
int i;
```

```
char ch[2];
```

```
};
```

```
struct cx s1;
```

The Variable s1 will occupy 4 bytes in memory and is represented as



The above datatype, if declared as union then

union ex

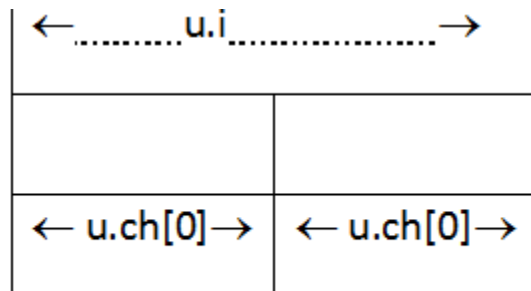
{

int i;

char ch[2];

}

union ex u;



Program to illustrate the use of unions

```
#include<stdio.h>
main()
{
    union example
    {
        int i;
        char ch[2];
    };
    union exaple u;
    u.i = 412;
    print("\n u.i = %d",u.i);
    print("\n u.ch*0+ = %d",u.ch*0+);
    print("\n u.ch*1+ = %d",u.ch*1+);
    u.ch[0] = 50; /* Assign a new value */
    print("\n\n u.i = %d",u.i);
    print("\n u.ch*0+ = %d",u.ch[0]); print("\n u.ch*1+ = %d",u.ch*1+);
}
```

Output :

u.i = 512 u.ch[0] = 0 u.ch[1] = 2

u.i = 562 u.ch[0] = 50 u.ch[1] = 2

A union may be a member of a structure, and a structure may be a member of a union. And also structures and unions may be freely mixed with arrays.

Program to illustrate union with in a Structure

```
#include<stdio.h>
main()
{
    union id
    {
        char color;
    }
    int size;
};
    struct {
        char supplier[20];
        float cost;
        union id desc;
    } pant, shirt;
    printf("%d\n", sizeof(union id));
    shirt.desc.color = 'w';
    printf("%c %d\n", shirt.desc.color, shirt.desc.size);
    shirt.desc.size = 12;
    printf("%c %d\n", shirt.desc.color, shirt.desc.size);
}
```

Binary File Handling

- Binary File Handling is a process in which we create a file and store data in its original format. It means that if we stored an integer value in a binary file, the value will be treated as an integer rather than text.
- Binary files are mainly used for storing records just as we store records in a database. It makes it easier to access or modify the records easily.

Contd..

Mode	Description
rb	Open file in binary mode for reading only.
wb	Open file in binary mode for writing only. It creates the file if it does not exist. If the file exists, then it erases all the contents of the file.
ab	Open file in binary mode for appending data. Data is added to the end of the file. It creates the file if it does not exist.
rb+	Open file in binary mode for both reading and writing.
wb+	Open file in binary mode for both reading and writing. It creates the file if it does not exist. If the file exists, then it erases all the contents of the file.
ab+	Open a file in binary mode for reading and appending data. Data is added to the end of the file. It creates the file if it does not exist.

Reading and Writing a File

- For reading and writing to a text file, we use the functions `fprintf()` and `fscanf()`.
- They are just the file versions of `printf()` and `scanf()`. The only difference is that `fprintf()` and `fscanf()` expects a pointer to the structure `FILE`.

Writing a binary file using fwrite()

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    int num;
    FILE *fptr;

    // use appropriate location if you are using MacOS or Linux
    fptr = fopen("C:\\\\program.txt","w");
    if(fptr == NULL)
    {
        printf("Error!");
        exit(1);
    }
    printf("Enter num: ");
    scanf("%d",&num);
    fprintf(fptr,"%d",num);
    fclose(fptr);
    return 0;
}
```

This program takes a number from the user and stores in the file program.txt.

After you compile and run this program, you can see a text file program.txt created in C drive of your computer. When you open the file, you can see the integer you entered.

Read from a binary file using fread()

```
include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct threeNum
```

```
{
```

```
    int n1, n2, n3;
```

```
};
```

```
int main()
```

```
{
```

```
    int n;
```

```
    struct threeNum num;
```

```
    FILE *fptr;
```

```
    if ((fptr = fopen("C:\\program.bin", "rb")) == NULL){
```

```
        printf("Error! opening file");
```

Contd..

```
// Program exits if the file pointer returns NULL.
```

```
    exit(1);
```

```
}
```

```
for(n = 1; n < 5; ++n)
```

```
{
```

```
    fread(&num, sizeof(struct threeNum), 1, fptr);
```

```
    printf("n1: %d\tn2: %d\tn3: %d\n", num.n1, num.n2, num.n3);
```

```
}
```

```
fclose(fptr);
```

```
return 0;
```

```
}
```

In this program, you read the same file "program.bin" and loop through the records one by one.

Important Questions

1. Explain how do you explain enum with example program
2. Explain about files and with it types of file processing. Define structure.
3. Explain the C syntax of structure and union declaration and initialization with example.
4. Program to illustrate the usage of a Structure.
5. Program to illustrate union with in a Structure
6. Implement structures to read, write and compute average marks and the students scoring above and below the average marks for a class of N students.
7. Define Input and Output operations of a file explain with an example.
8. Difference between Struct and Enum in C with examples.
9. Given 2 university information files "name.txt" and "usn.txt". write a program to copy the contents to "output.txt" from both the files and also display the content on the output screen.
10. Writing a binary file using fread() and fwrite()

Case Study

- System to manage different types of geometric shapes, including rectangles and circles. The system needs to calculate the area and perimeter of each shape.
- Imagine you are developing a simple program to manage a list of contacts. The program should be able to read contacts from a file, add new contacts and save the new contacts